Aula 5 - Product, Process and Rescource Attributes

Prof. Danilo de Quadros Maia Filho

Departamento:
Engenharia de Software

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

Objetivos da Aula

O que vamos aprender:

- Equações e definições de métricas utilizadas tanto pela comunidade científica quanto pelo mercado
- Atributos diversos de Produto, Recurso e Processo
- Atributos Internos de Produto
- Atributos Externos de Produto
- Métricas diversas já padronizadas e aceitas pela literatura
- O modelo de qualidade da ISO/IEC 25010, a fundação do time (Team Topologies) e os benchmarks DORA
- Um problema atual: o que "Size"significa quando o código é gerado por IA

Pontos de Função e Complexidade Ciclomática têm aulas próprias – 6

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

Size

Métricas de tamanho mostram o quanto de determinada entidade nós temos.

Existem limitações em medir o tamanho:

> *. . . there is a major problem with the lines-of-code measure: it is not consistent because some lines are more difficult to code than others . . . . One solution to the problem is to give more weight to lines that have more 'stuff' in them. (CONTE ET AL. 1986)*

Size

Size é utilizado para calcular atributos derivados. Também utilizado para atributos de planejamento de projetos.

Exemplos:

$$Productivity = Size/Effort \tag{1}$$

$$Defect\ density = Defect\ count/Size \tag{2}$$

Embora uma métrica em si possa não ter informações suficientes para tomadas de decisão, não a rejeitamos. Ela vai compor um grupo de métricas.

Size

Conhecer o tamanho da maioria dos produtos de desenvolvimento pode ser muito útil. O tamanho de uma especificação de requisitos pode prever o tamanho e a complexidade de um projeto, o que, por sua vez, pode prever o tamanho do código. Os atributos de tamanho do projeto e do código podem determinar o esforço necessário para os testes, bem como o esforço exigido para adicionar novas funcionalidades. As medidas de tamanho são comumente usadas para indicar a quantidade de reutilização em um sistema.

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

Code Size

Devemos ter muito cuidado para esclarecer *o que estamos contando* e *como estamos contando*. Em particular, devemos explicar como cada um dos seguintes itens é tratado:

- Linhas em branco
- Linhas de comentário
- Declarações de dados
- Linhas que contêm várias instruções separadas

...uma contagem pode ser até cinco vezes maior do que outra, simplesmente por causa da diferença na técnica de contagem (Jones, 2008).

Code Size

There is some general consensus that blank lines and comments should not be counted. Conte et al. define an LOC as any line of program text that is not a comment or blank line, regardless of the number of statements or fragments of statements on the line. This definition specifically includes all lines containing program headers, declarations, and executable and nonexecutable statements (Conte et al. 1986). Grady and Caswell report that Hewlett-Packard defines an LOC as a noncommented source statement: any statement in the program except for comments and blank lines (Grady and Caswell 1987).

Exemplo: Code Size no projeto Linux

Another use of NCLOC is to evaluate the growth of systems over time. Godfrey and Tu used NCLOC analyzed the growth of the Linux system and key subsystems over multiple releases. They found that Linux had been growing at a superlinear rate (Godfrey and Tu 2000).

Exemplo: Code Size no projeto Linux

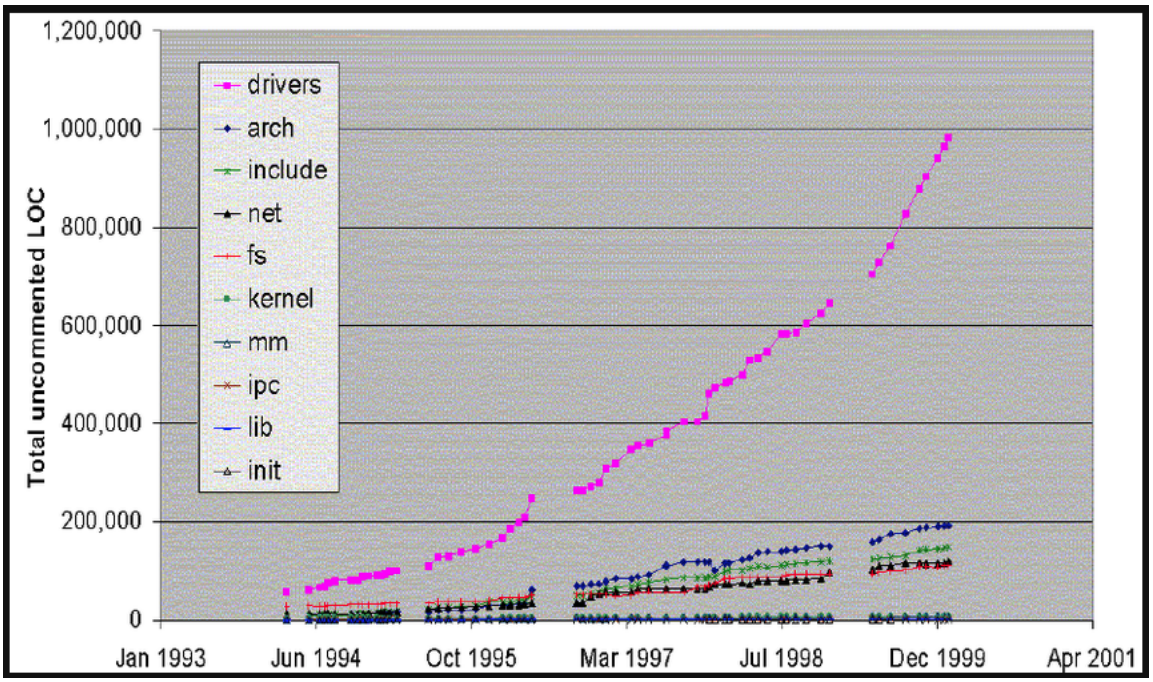


Figura 1: Exemplo: Linux (Godfrey and Tu 2000)

Exemplo: Code Size no projeto ARLA

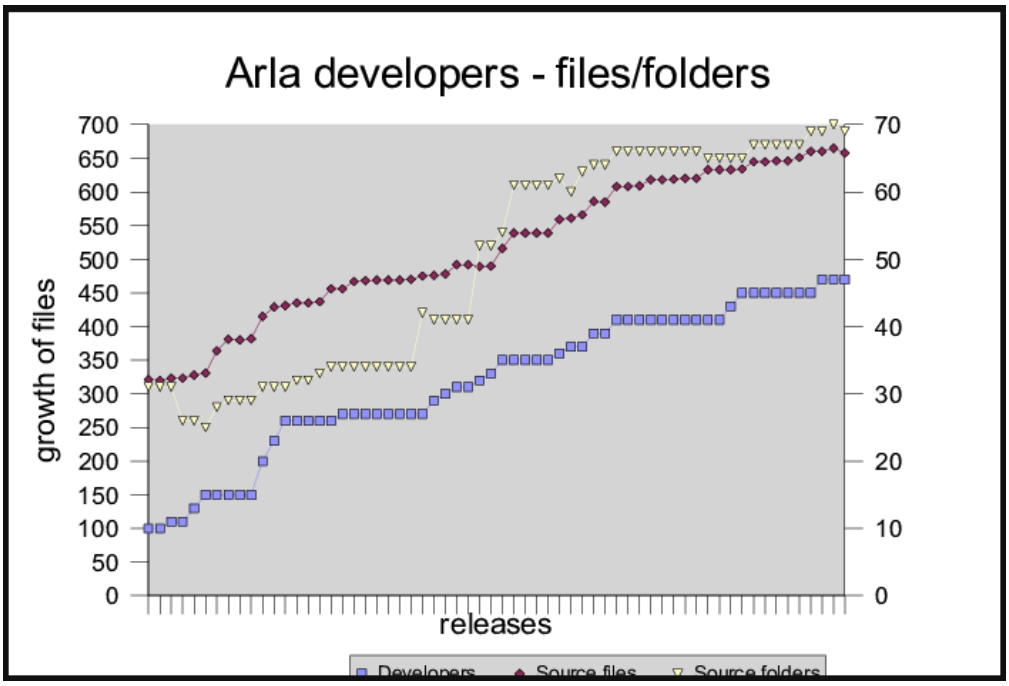


Figura 2: Exemplo: Code Size versus Developers

Code Size

De certo modo, informações valiosas sobre o tamanho são perdidas quando as linhas de comentário não são contadas. Em muitas situações, o tamanho do programa é importante para decidir quanto espaço de armazenamento será necessário para o código-fonte, ou quantas páginas serão necessárias para uma impressão. Nesse caso, o tamanho do programa deve refletir as linhas em branco e as linhas comentadas.

Code Size: Equations

$$Total\ size(LOC) = NCLOC + CLOC \tag{3}$$

$$Density\ of\ comments = \frac{CLOC}{LOC} \tag{4}$$

$$LOC_{total} = LOC_{exec} + LOC_{decl} + LOC_{coment} + LOC_{branco} \tag{5}$$

$$LOC_{exec} = \sum_{i=1}^{n} \text{instruções executáveis} \tag{6}$$

$$LOC_{decl} = LOC_{dados} + LOC_{diretivas} \tag{7}$$

Code Size: Equations

$$LOC_{efetivo} = LOC_{total} - (LOC_{coment} + LOC_{branco}) \quad (8)$$

$$Densidade_{coment} = \frac{LOC_{coment}}{LOC_{total}} \quad (9)$$

$$Reuso = \frac{LOC_{reutilizado}}{LOC_{total}} \quad (10)$$

$$Produtividade = \frac{LOC_{entregues}}{Esforco} \quad (11)$$

Code Size: Executable Statements (ES)

Instruções Executáveis: O foco está em medir *quanto de lógica efetiva foi escrita e pode ser executada pela máquina*. Não considera:

- Linhas em branco
- Linhas de comentários
- Diretivas de compilador
- Declarações de dados (dependendo da convenção adotada)

Quantos LOC executáveis entregues por hora de programação?

$$LOC_{ES} = \sum_{i=1}^{n} \text{instruções executáveis} \tag{12}$$

Code Size: Delivered Source Instructions (DSI)

Instruções-fonte entregues: O foco está em medir o *produto entregue ao cliente*. Não considera:

- Linhas de comentários
- Linhas em branco
- Código descartado (não entregue na versão final)

Considera:

- Código executável
- Declarações não executáveis
- Diretivas de compilador

$$LOC_{DSI} = LOC_{exec} + LOC_{decl} + LOC_{diretivas} \tag{13}$$

Exemplo: code size em um projeto COBOL

Uma grande organização do Reino Unido escreveu o código de uma única aplicação em dois tipos diferentes de COBOL. Embora o código execute os mesmos tipos de funções, a diferença na densidade de comentários é marcante, conforme mostrado abaixo:

	Number of Programs	Total Lines of Code	Comment Density (%)
Batch COBOL	335	670,000	16
CICS COBOL	273	507,000	26

Figura 3: Exemplo: Comparação entre métricas LOC de software

Exemplo: Checklist do SEI

O *Software Engineering Institute* realizou um estudo na *Defense Information Systems Agency* (Rozum e Florac, 1995). Linhas de comentário, em branco e códigos removidos não foram contados. O que foi considerado:

- Todo código executável
- Declarações não executáveis e diretivas do compilador

Exemplo: Checklist do SEI

Possíveis origens para geração do código:

- Programado
- Gerado com geradores de código-fonte
- Convertido com tradutores automáticos
- Copiado ou reutilizado sem alteração
- Modificado

Statement type	*Include*	*Exclude*
Executable		
Nonexecutable		
Declarations		
Compiler directives		
Comments		
On their own lines		
On lines with source code		
Banners and nonblank spacers		
Blank (empty) comments		
Blank lines		

How produced	*Include*	*Exclude*
Programmed		
Generated with source code generators		
Converted with automatic translators		
Copied or reused without change		
Modified		
Removed		

Origin	*Include*	*Exclude*
New work: no prior existence		
Prior work: taken or adapted from		
A previous version, build, or release		
Commercial, off-the-shelf software, other than libraries		
Government furnished software, other than reuse libraries		
Another product		
A vendor-supplied language support library (unmodified)		
A vendor-supplied operating system or utility (unmodified)		
A local or modified language support library or operating system		
Other commercial library		
A reuse library (software designed for reuse)		
Other software component or library		

Figura 4: Exemplo: Check list proposto pelo checklist Software Engineering Institute

Exemplo: Code Size em um script

```
with TEXT_IO; use TEXT_IO;
procedure Main is

       --This program copies characters from an input
       --file to an output file. Termination occurs
       --either when all characters are copied or
       --when a NULL character is input

       NullchAr, Eof: exception;
       Char: CHARACTER;
       Input_file, Ouptu_file, Console: FILE_TYPE;
Begin
       loop
              Open (FILE => Input_file, MODE => IN_FILE,
                            NAME => "CharsIn");
              Open (FILE => Output_file, MODE = > OUT_FILE,
                            NAME => "CharOut");
              Ge (Input_file, Char);
              if END_OF_FILE (Input_file) then
                       raiseEof;
              elseif Char = ASCII.NUL then
                       raiseNullchar;
              else
                       Put(Output_file, Char);
              endif;
       end loop;
exception
       whenEof => Put (Console, "no null characters");
       whenNullchar => Put (Console, "null terminator");
end Main
```

Figura 5: Exemplo: Código

Exemplo: Code Size em um script

30	Count of the number of physical lines (including blank lines)
24	Count of all lines except blank lines and comments
20	Count of all statements except comments (statements taking more than one line count as only one line)
17	Count of all lines except blank lines, comments, declarations and headings
13	Count of all statements except blank lines, comments, declarations and headings
6	Count of only the ESs, not including exception conditions

Figura 6: Exemplo: Contagem

Code Size - Alternativas ao LOC

Alternativas que podem ser nativamente computadas por qualquer máquina/software, e refletem apropriadamente o tamanho de um software:

- number of bytes
- number of characters

Code Size - Alternativas ao LOC

Se α é o número médio de caracteres por linha de texto do programa, então temos o reescalonamento

$$CHAR = \alpha \cdot LOC \tag{14}$$

o que expressa uma relação estocástica entre *LOC* e *CHAR*.

De forma semelhante, podemos usar qualquer múltiplo constante das medidas propostas como uma medida alternativa válida de tamanho. Assim, podemos usar *KLOC* (milhares de linhas de código) ou *KDSI* (milhares de instruções-fonte entregues) para medir o tamanho de um programa.

Code Size: Diferença entre linguagens

O número de *Lines of Code* (LOC) para implementar a **mesma funcionalidade** varia entre linguagens por fatores como: expressividade da linguagem, sintaxe, idiomas e bibliotecas disponíveis, paradigma (imperativo, OO, funcional), geradores de código, *frameworks* e, ainda, pelas **regras de contagem** adotadas (inclusão/exclusão de comentários, linhas em branco, diretivas, declarações).

Code Size: Diferença entre linguagens

Modelo por fator de linguagem

Um modo simples de comparar tamanhos entre linguagens é assumir um fator de proporcionalidade κ_L ("verbosidade") específico da linguagem L:

$$\mathrm{LOC}^{(L)} \;\approx\; \kappa_L \,\mathrm{FP} \;+\; \varepsilon, \tag{15}$$

onde FP são *Function Points* (ou outra medida funcional) e ε agrega variações do domínio, estilo e regras de contagem. A partir de (15), podemos converter LOC entre duas linguagens L_1 e L_2 para a **mesma funcionalidade**:

$$\mathrm{LOC}^{(L_2)} \;\approx\; \frac{\kappa_{L_2}}{\kappa_{L_1}}\,\mathrm{LOC}^{(L_1)}. \tag{16}$$

Code Size: Diferença entre linguagens

LOC normalizado (comparação justa)

Para comparações independentes da linguagem, pode-se normalizar LOC por κ_L:

$$\mathrm{NLOC} = \frac{\mathrm{LOC}^{(L)}}{\kappa_L}, \tag{17}$$

de modo que dois artefatos com a mesma funcionalidade tenham NLOC semelhantes, a despeito da linguagem utilizada.

Code Size: Diferença entre linguagens

Relação com caracteres por linha

Como diferentes linguagens e estilos produzem linhas mais longas ou curtas, é útil relacionar LOC com o total de caracteres:

$$\mathrm{CHAR}^{(L)} = \alpha_L \, \mathrm{LOC}^{(L)}, \tag{18}$$

onde α_L é o número médio de caracteres por linha na linguagem L.

Code Size: Ferramentas CASE

Atualmente, pedaços de código e programa inteiro são construídos com ferramentas não textuais, também chamadas de ferramentas *low code*.

Menus, botões, cabeçalhos, footers, etc, tudo isso pode ser construído sem programar uma linha de código sequer.

Como mensurar isto no Size do seu programa? Por exemplo, alguns KB ou até MB de programa podem ser gerados dessa forma. Como identificar ou mesmo ponderar o que é Size e o que não é?

Code Size: o mesmo problema, versão 2025

O problema das ferramentas *low-code* voltou – em nova roupagem – com assistentes de IA (Copilot, Cursor, Claude Code): boa parte do LOC de um repositório hoje pode ser *sugerida pela IA e apenas aceita* pelo desenvolvedor.

Isso quebra silenciosamente uma suposição implícita em quase toda métrica derivada de Size:

$$Productivity = \frac{LOC_{entregues}}{Esforco_{humano}} \tag{19}$$

Se uma fração p do LOC entregue foi gerada por IA, **o numerador não caiu, mas o Esforço Humano no denominador pode ter caído muito mais** – inflando artificialmente a produtividade aparente. A métrica não ficou errada; a **entidade que ela representa** mudou.

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

Design Size

De forma similar ao Code Size, no Design Size algumas medidas de tamanho poderão ser adotadas:

- **Pacotes**: Número de subpacotes, número de classes, interfaces (Java) ou classes abstratas (C++).
- **Métodos ou operações**: Número de parâmetros, número de versões sobrecarregadas de um método ou operação.

Design Size

- **Padrões de projeto**:
  - Número de diferentes padrões de projeto usados em um design.
  - Número de realizações de padrões de projeto para cada tipo de padrão.
  - Número de classes, interfaces ou classes abstratas que desempenham papéis em cada realização de padrão.
- **Classes, interfaces ou classes abstratas**: Número de métodos ou operações públicas, número de atributos.

Design Size

Uma das primeiras medidas de tamanho em projeto orientado a objetos foi a métrica de métodos ponderados por classe (WMC, do inglês Weighted Methods per Class) (Chidamber e Kemerer, 1994). Conforme proposto, o WMC é medido somando os pesos dos métodos em uma classe, onde os pesos são fatores de complexidade não especificados para cada método. O peso usado na maioria dos estudos é 1. Com peso igual a 1, o WMC torna-se uma medida de tamanho — o número de métodos em uma classe.

Design Size

Por exemplo, usando peso igual a 1, Zhou e Leung descobriram que o WMC era uma das variáveis independentes que podiam prever de forma eficaz a severidade de falhas em dados de um sistema em C++ da NASA (Zhou e Leung, 2006). Outro estudo, de Pai e Dugan, aplicou uma análise bayesiana para desenvolver um modelo causal relacionado a medidas de projeto orientado a objetos, incluindo duas medidas de tamanho — WMC (novamente usando peso igual a 1) e o número de LOCs — juntamente com várias medidas de estrutura orientada a objetos (a serem discutidas no próximo capítulo) — para prever a propensão a falhas (Pai e Dugan, 2007).

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

Requirement Size

De forma similar ao Code Size e ao Design Size, algumas medidas de tamanho poderão ser adotadas para os atributos de Requisitos.

A seguir serão apresentados exemplos de elementos atômicos envolvendo modelos de requisitos e especificação:

1. **Use case diagrams:** Number of use cases, actors, and relationships of various types.
2. **Use case:** Number of scenarios, size of scenarios in terms of steps, or activity diagram model elements.

Requirement Size

3. **Domain model (expressed as a UML class diagram):** Number of classes, abstract classes, interfaces, roles, operations, and attributes.
4. **UML OCL specifications:** Number of OCL expressions, OCL clauses.
5. **Alloy models:** Number of alloy statements—signatures, facts, predicates, functions, and assertions (Jackson 2002).

Requirement Size

6. **Data-flow diagrams used in structured analysis and design:** Processes (bubble nodes), external entities (box nodes), data-stores (line nodes) and data-flows (arcs).
7. **Algebraic specifications:** Sorts, functions, operations, and axioms.
8. **Z specifications:** The various lines appearing in the specification, which form part of either a type declaration or a (nonconjunctive) predicate (Spivey 1993).

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

Functional Size

Diferente do Code Size, o **tamanho funcional** mede *o que o sistema faz do ponto de vista do usuário* – é independente de linguagem, plataforma ou estilo de codificação. Pode ser estimado antes de uma única linha de código ser escrita.

Família de métodos de FSM (*Functional Size Measurement*, normatizados pela ISO/IEC 14143):

- **Function Point Analysis (FPA/IFPUG)** – o método clássico, baseado em entradas, saídas, consultas e arquivos lógicos
- **COSMIC** – baseado em movimentos de dados (entrada, saída, leitura, escrita), mais adequado a sistemas em tempo real
- **Use Case Points** – deriva tamanho a partir de casos de uso e atores

O cálculo detalhado de Pontos de Função – com exemplo passo a passo – é o tema central da **Aula 6**.

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

ISO/IEC 25010 — o Modelo Completo

A norma define **9 características** de Qualidade de Produto (revisão 2023), cada uma com subcaracterísticas próprias:

Adequação Funcional	Manutenibilidade
Eficiência de Desempenho	Portabilidade
Compatibilidade	Segurança
Capacidade de Interação *(ex-Usabilidade)*	Segurança Funcional *(Safety, nova)*
Confiabilidade	

Hoje vamos aprofundar métricas para **Confiabilidade** – as demais características seguem a mesma lógica: definir a subcaracterística, depois derivar uma métrica mensurável para ela.

ISO/IEC 25010 — Qualidade de Produto

Taxa de falhas (Reliability - Fault Rate): mede a frequência de falhas por unidade de tempo.

$$\text{FaultRate} = \frac{N_f}{T} \tag{20}$$

Onde N_f é o número de falhas observadas durante o período de teste, e T é o tempo total em que o sistema foi submetido a operação ou teste.

ISO/IEC 25010 — Qualidade de Produto

Tempo médio entre falhas (MTBF) (Reliability):

$$\text{MTBF} = \frac{T_{\text{op}}}{N_f} \tag{21}$$

Onde T_{op} é o tempo total de operação sem falhas acumulado, e N_f é o número de falhas ocorridas no período.

ISO/IEC 25010 — Qualidade de Produto

Tempo médio para restaurar (MTTR) (Reliability):

$$\text{MTTR} = \frac{\sum t_{\text{repair}}}{N_r} \tag{22}$$

Onde t_{repair} são os tempos de reparo de cada incidente, e N_r é o número total de incidentes resolvidos.

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

ISO/IEC 15939 — Processo de Medição

Esta norma foca na definição de um modelo de medição estruturado, não apresenta equações diretas, mas define conceitos.

A **Precisão da Medição (Measurement Accuracy)** pode ser compreendido como a proximidade entre o valor medido e o valor real. Embora não haja fórmula prescrita, pode-se representar como:

$$\text{Accuracy} = 1 - \frac{|\text{ValorMedido} - \text{ValorVerdadeiro}|}{\text{ValorVerdadeiro}} \tag{23}$$

Onde "ValorMedido" é o valor obtido pela métrica, e "ValorVerdadeiro" é o valor ideal ou de referência.

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

ISO/IEC 1061 — Métricas de Qualidade de Software

Embora esta norma descreva um processo para definir métricas, uma métrica típica recomendada é:

Densidade de defeitos por trecho de código (Defect Density):

$$\text{DefDensity} = \frac{N_d}{KLOC} \tag{24}$$

Onde N_d é o número de defeitos detectados durante o ciclo de vida, e $KLOC$ é o tamanho do código em milhares de linhas.

ISO/IEC 1061 — Métricas de Qualidade de Software

Índice de modularidade (Modularity Index) — um exemplo conceitual:

$$\text{MI} = \frac{1}{M} \sum_{i=1}^{M} \left(1 - \frac{C_i}{C_{\text{max}}}\right) \tag{25}$$

Onde M é o número de módulos, C_i é o número de acoplamentos do módulo i, e C_{max} é o valor máximo aceitável de acoplamento. (Essa métrica é inspirada nos princípios da norma.)

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

IEEE Std 982.1 (1988) - Produto

Este padrão define métricas clássicas de produto como:

Taxa de defeitos por requisito (Defects per Requirement):

$$\text{DPR} = \frac{N_d}{N_r} \tag{26}$$

Onde N_d é o número de defeitos identificados relacionados aos requisitos, e N_r é o número de requisitos especificados.

IEEE Std 982.1 (1988) - Produto

Cobertura de Testes de Requisitos (Requirements Test Coverage):

$$\text{RTC} = \frac{N_t}{N_r} \times 100\% \tag{27}$$

Onde N_t é o número de requisitos efetivamente testados, e N_r é o total de requisitos.

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

ISO/IEC 12207 — Processo

Norma de processos do ciclo de vida de software (aquisição, desenvolvimento, operação, manutenção, apoio). Foco em *conformidade de processos*; não define fórmulas numéricas.

ISO/IEC 12207 — Processo

Índice de conformidade processual

$$\mathrm{PCR} = \frac{\textit{resultados/artefatos conformes}}{\textit{resultados/artefatos avaliados}} \tag{28}$$

Uso: auditorias internas/externas; **não** é métrica padronizada na norma.

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

SPICE/ISO/IEC 15504 (330xx) — Processo

Avaliação de capacidade de processo por atributos (PA) em níveis (0–5), com julgamentos como N, P, L, F (Não, Parcialmente, Largamente, Totalmente alcançado). A norma define método de avaliação, não fórmulas universais.

SPICE/ISO/IEC 15504 (330xx) — Processo

Grau de atendimento de um atributo de processo

$$\mathrm{AA} = \frac{\textit{indicadores atendidos}}{\textit{indicadores avaliados}} \tag{29}$$

Onde: "indicadores" são evidências por atributo (p.ex., Process Attributes 2.1, 2.2).

Nota: A decisão de nível usa regras de julgamento da norma; a razão acima é apenas apoio quantitativo.

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

CMMI

Modelo de maturidade/capacidade de processos (níveis 1–5) baseado em práticas específicas e genéricas. A avaliação é qualitativa; não há equação normativa oficial de “maturidade”.

CMMI — medida derivada (não normativa)

Cobertura de práticas implementadas

$$\mathrm{CC} = \frac{\#\text{práticas implementadas}}{\#\text{práticas requeridas}} \tag{30}$$

Uso: acompanhamento interno de adoção; **não** substitui avaliação formal (SCAMPI).

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

COCOMO (Constructive Cost Model)

Modelo de estimativa de esforço e custo de projetos de software. A versão moderna, **COCOMO II**, incorpora fatores de escala e multiplicadores de esforço para aumentar a precisão da previsão.

COCOMO II — Esforço

Definição: O esforço em pessoa-meses necessário para desenvolver o software.

$$E = A \cdot (\text{Size})^{B} \cdot \prod_{i=1}^{n} \text{EM}_i \tag{31}$$

Onde: Size = tamanho do software em KSLOC (ou equivalente); A, B = constantes calibradas; EM_i = multiplicadores de esforço (p.ex., RUSE, RELY, CPLX, ACAP, PCAP).

Interpretação: O esforço cresce de forma exponencial com o tamanho do software, ajustado por fatores de ambiente, equipe e requisitos.

COCOMO II — Expoente de Escala

Definição: O expoente B representa economias ou deseconomias de escala conforme o projeto cresce.

$$B = b + 0.01 \cdot \sum_{j=1}^{5} \mathrm{SF}_j \tag{32}$$

Onde: $b \approx 0.91$ (valor típico publicado); SF_j = fatores de escala (PREC, FLEX, RESL, TEAM, PMAT).

Interpretação: Quanto maiores os fatores de escala, maior o valor de B, indicando crescimento mais que linear no esforço requerido.

COCOMO II — Cronograma

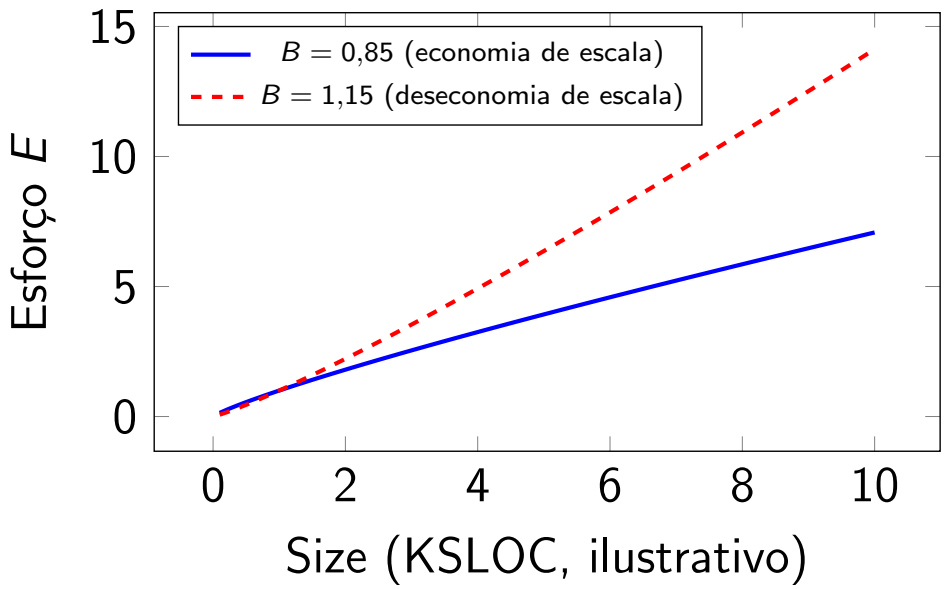
Definição: Estimativa do cronograma de desenvolvimento do projeto (tempo em meses).

$$\mathrm{TDEV} = C \cdot E^F \tag{33}$$

Onde: C, F = constantes calibradas do modelo; E = esforço em pessoa-meses.

Interpretação: O tempo de desenvolvimento cresce de forma sublinear em relação ao esforço, refletindo limites práticos de paralelização de atividades.

COCOMO II — Visualizando B



Com $A = 1$ fixo (curvas normalizadas): quando $B < 1$, dobrar o tamanho *menos que dobra* o esforço (equipes maduras, processos bem definidos); quando $B > 1$, dobrar o tamanho *mais que dobra* o esforço (coordenação, comunicação, retrabalho crescem junto).

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

McCall's Quality Model - Produto

Modelo clássico de qualidade de produto com fatores (p.ex., correção, confiabilidade, eficiência, integridade, usabilidade, manutenibilidade) e critérios/métricas associadas. Não há fórmula única normativa.

McCall's Quality Model — Produto

Índice de qualidade por fator (composição linear)

$$Q_f = \sum_{j=1}^{k} w_j \, m_j \tag{34}$$

Onde: m_j = métrica normalizada associada ao fator f; w_j = peso $(\sum w_j = 1)$.

Observação: forma *didática*, não prescrita pela literatura original como norma obrigatória.

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

CK Metrics (Chidamber & Kemerer)

Os CK Metrics (Chidamber & Kemerer) são...

um conjunto clássico de seis métricas para software orientado a objetos, proposto em 1994, que avaliam propriedades internas de classes como complexidade (WMC, RFC), herança e reuso (DIT, NOC), acoplamento (CBO) e coesão (LCOM). Seu objetivo é oferecer uma base formal para medir atributos de qualidade em sistemas OO, apoiando manutenção, evolução e reusabilidade.

Weighted Methods per Class (WMC)

Definição: Soma das complexidades dos métodos de uma classe.
Interpretação: Classes com WMC alto são mais complexas, difíceis de manter e testar. Classes com WMC baixo são mais simples e reutilizáveis.

$$\mathrm{WMC} = \sum_{i=1}^{m} c_i \tag{35}$$

Onde: $m = \#$métodos da classe; $c_i =$ complexidade do método i (p.ex., ciclomat. de McCabe).

CK Metrics

Depth of Inheritance Tree (DIT)

Definição: Profundidade máxima da árvore de herança de uma classe até a raiz.
Interpretação: DIT alto indica maior reutilização por herança, mas pode aumentar a dificuldade de entendimento e teste. DIT baixo sugere simplicidade, porém com menor reuso.

$$\text{DIT} = \text{número de ancestrais na árvore de herança} \tag{36}$$

CK Metrics

Number of Children (NOC)

Definição: Número de subclasses imediatas de uma classe.
Interpretação: NOC alto mostra que a classe é muito reutilizada, mas mudanças nela podem impactar muitas subclasses. NOC baixo sugere pouco reuso e menor risco de propagação de erros.

$$\text{NOC} = \text{número de subclasses imediatas} \tag{37}$$

CK Metrics

Coupling Between Objects (CBO)

Definição: Número de classes às quais uma classe está acoplada (por uso de métodos, atributos ou herança).
Interpretação: CBO alto significa forte dependência e baixa reusabilidade. CBO baixo implica maior independência e facilidade de manutenção.

$$\text{CBO} = \text{número de classes às quais esta classe está acoplada} \quad (38)$$

CK Metrics

Response For a Class (RFC)

Definição: Número de métodos que podem ser potencialmente executados em resposta a uma mensagem recebida pela classe.
Interpretação: RFC alto indica maior complexidade de comportamento, tornando a classe mais difícil de prever e testar. RFC baixo sugere comportamento simples e previsível.

$$\mathrm{RFC} = |RS| \tag{39}$$

Onde: RS = conjunto de métodos da classe $\cup$ métodos por ela invocados.

Lack of Cohesion in Methods (LCOM, forma original)

Definição: Mede a falta de coesão entre os métodos de uma classe, ou seja, o quanto eles compartilham atributos.
Interpretação: LCOM alto indica baixa coesão (a classe pode estar assumindo muitas responsabilidades). LCOM baixo, próximo de zero, indica boa coesão interna.

$$\text{LCOM} = \begin{cases} P - Q, & \text{se } P > Q \\ 0, & \text{caso contrário} \end{cases} \tag{40}$$

Onde: P = #pares de métodos que *não* compartilham atributos; Q = #pares que compartilham.

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

Métricas de Halstead

As métricas de Halstead foram propostas por Maurice Halstead em 1977 no livro **"Elements of Software Science"**. A ideia central é que um programa pode ser visto como um conjunto de operadores e operandos, e a partir de contagens simples desses elementos é possível derivar medidas de tamanho, complexidade, esforço e até de qualidade.

Métricas de Halstead

Elementos básicos

$$n_1 = \text{número de operadores distintos}$$

$$n_2 = \text{número de operandos distintos}$$

$$N_1 = \text{número total de ocorrências de operadores}$$

$$N_2 = \text{número total de ocorrências de operandos}$$

Métricas de Halstead

Vocabulário:

$$n = n_1 + n_2 \tag{41}$$

Comprimento:

$$N = N_1 + N_2 \tag{42}$$

Volume:

$$V = N \cdot \log_2(n) \tag{43}$$

Dificuldade:

$$D = \frac{n_1}{2} \cdot \frac{N_2}{n_2} \tag{44}$$

Métricas de Halstead

Esforço:

$$E = D \cdot V \tag{45}$$

Tempo estimado:

$$T = \frac{E}{18} \tag{46}$$

Defeitos estimados:

$$B = \frac{V}{3000} \tag{47}$$

Métricas de Halstead

Interpretação:

Quanto maior o vocabulário (n) e o comprimento (N), maior a complexidade cognitiva.

Métricas como D e E tentam quantificar o esforço humano necessário.

Apesar de limitações (linguagens diferentes, contagens dependentes da ferramenta), a família de métricas de Halstead ainda é usada em análise estática de código e em estudos de produtividade e qualidade.

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

Fenton & Pfleeger (1990s, 2010s)

Probabilidade de falha em um intervalo de tempo
Mede a confiabilidade como a probabilidade de o sistema não falhar em um intervalo t.

$$R(t) = e^{-\lambda t} \tag{48}$$

Onde $R(t)$ é a confiabilidade até o tempo t, λ é a taxa de falhas assumida como constante, e t é o tempo de operação.

Fenton & Pfleeger (1990s, 2010s)

Disponibilidade

Percentual de tempo em que o sistema está operacional e acessível quando requerido.

$$A = \frac{MTBF}{MTBF + MTTR} \tag{49}$$

Onde A é a disponibilidade, $MTBF$ é o tempo médio entre falhas, e $MTTR$ é o tempo médio para reparo.

Fenton & Pfleeger (1990s, 2010s)

Densidade de defeitos normalizada
Proporção de defeitos encontrados por unidade de tamanho funcional.

$$DD = \frac{N_d}{FP} \tag{50}$$

Onde DD é a densidade de defeitos, N_d é o número de defeitos identificados e FP é o número de Function Points entregues.

Fenton & Pfleeger (1990s, 2010s)

Índice de confiabilidade operacional

Confiabilidade calculada considerando tanto falhas quanto tempo de execução.

$$OR = 1 - \frac{N_f}{T_e} \tag{51}$$

Onde OR é o índice de confiabilidade operacional, N_f é o número de falhas observadas e T_e é o total de execuções ou operações monitoradas.

Fenton & Pfleeger (1990s, 2010s)

Métrica de acoplamento

Relaciona o número de dependências externas com o total de módulos do sistema.

$$C = \frac{\sum_{i=1}^{M} D_i}{M} \tag{52}$$

Onde C é o acoplamento médio, M é o número de módulos, e D_i é o número de dependências externas do módulo i.

Fenton & Pfleeger (1990s, 2010s)

Índice de coesão

Mede a proporção de inter-relações internas entre elementos de um módulo.

$$COH = \frac{I}{I + E} \tag{53}$$

Onde COH é a coesão, I é o número de interações internas entre elementos do módulo, e E é o número de interações externas.

Fenton & Pfleeger (1990s, 2010s)

Complexidade de dados

Mede o esforço adicional causado pela complexidade de estruturas de dados em uso.

$$DC = \sum_{j=1}^{S} w_j \cdot a_j \tag{54}$$

Onde DC é a complexidade de dados, S é o número de estruturas de dados distintas, w_j é o peso associado à complexidade da estrutura j, e a_j é o número de acessos ou operações sobre j.

Fenton & Pfleeger (1990s, 2010s)

Métrica de testabilidade
Proporção entre o número de casos de teste executados e o número de caminhos independentes do programa.

$$T = \frac{N_{tc}}{P} \tag{55}$$

Onde T é a testabilidade, N_{tc} é o número de casos de teste executados, e P é o número de caminhos independentes (ciclomática).

Fenton & Pfleeger (1990s, 2010s)

Confiabilidade prevista (modelo de Musa)
Modelo estatístico para prever falhas remanescentes com base em tempo de execução.

$$\lambda(t) = \lambda_0 e^{-\theta t} \tag{56}$$

Onde $\lambda(t)$ é a taxa de falhas no tempo t, λ_0 é a taxa inicial de falhas, e θ é o parâmetro de decaimento (constante positiva).

Fenton & Pfleeger (1990s, 2010s)

Qualidade percebida
Mede a qualidade final como função de fatores de defeito, confiabilidade e eficiência.

$$Q = w_1 \cdot \frac{1}{DD} + w_2 \cdot R(t) + w_3 \cdot A \tag{57}$$

Onde Q é o índice de qualidade, DD é a densidade de defeitos, $R(t)$ é a confiabilidade, A é a disponibilidade, e w_1, w_2, w_3 são pesos que refletem a importância relativa de cada dimensão.

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

DORA

DORA (DevOps Research and Assessment) é...

um modelo de avaliação de desempenho de equipes de desenvolvimento e operações, fundamentado em pesquisa empírica de larga escala conduzida por Forsgren, Humble e Kim.

Esse modelo é baseado em um conjunto de quatro métricas-chave que medem, de forma objetiva e comparável, a capacidade de entrega de software de uma organização em termos de velocidade e estabilidade.

DORA

Em termos formais, o modelo DORA busca equilibrar dois eixos fundamentais da engenharia de software:

- Velocidade de entrega (Lead Time, Deployment Frequency)
- Estabilidade do serviço (Change Failure Rate, MTTR)

DORA

Perspetiva de Processo?

As métricas DORA têm a ver principalmente com processo, mas também se conectam indiretamente com produto e resources.

Elas são indicadores de eficiência e maturidade do processo de entrega de software (ex.: tempo para implantar, frequência de deploys, taxa de falha, tempo de recuperação).

Portanto, o foco central é no processo.

DORA

Perspetiva de Produto?

Um processo mais ágil e confiável permite que o produto evolua mais rápido, com entregas frequentes e seguras.

Embora DORA não avalie diretamente a qualidade funcional do produto (se o usuário gosta, se resolve o problema), ele garante que o produto possa ser lançado e atualizado continuamente.

DORA

Perspetiva de Resource?

DORA não mede quantidade de pessoas, custo ou capacidade de infraestrutura.

Porém, os resultados podem influenciar decisões de alocação de recursos (ex.: aumentar automação, investir em observabilidade, melhorar pipeline de CI/CD).

DORA

As métricas DORA principais são quatro:

Lead Time for Changes (Tempo de Entrega de Mudanças)

- Mede quanto tempo leva desde o commit de código até a implantação em produção.
- Indica a agilidade da equipe em entregar valor.

$$LT = t_{\text{deploy}} - t_{\text{commit}} \tag{58}$$

onde t_{deploy} é o instante da implantação e t_{commit} é o instante do commit.

DORA

Deployment Frequency (Frequência de Deploys)

- Mede com que frequência a equipe realiza deploys em produção.
- Frequências altas estão associadas a maior capacidade de inovação e resposta rápida ao mercado.

$$DF = \frac{N_{\text{deploys}}}{\Delta t} \tag{59}$$

onde N_{deploys} é o número de implantações em um período Δt.

DORA

Change Failure Rate (Taxa de Falha de Mudanças)

- Mede a porcentagem de deploys que causam incidentes, rollback ou exigem correções urgentes.
- Avalia a qualidade do processo de entrega.

$$CFR = \frac{N_{\text{falhas}}}{N_{\text{deploys}}} \tag{60}$$

onde N_{falhas} é o número de implantações que resultaram em incidentes ou rollback.

DORA

Mean Time to Restore (MTTR – Tempo Médio de Recuperação)

- Mede quanto tempo a equipe leva para restaurar o serviço após uma falha em produção.
- Reflete a resiliência operacional.

$$MTTR = \frac{\sum_{i=1}^{N} \left(t_{\text{recuperação},i} - t_{\text{falha},i} \right)}{N} \tag{61}$$

onde $t_{\text{falha},i}$ é o instante da falha, $t_{\text{recuperação},i}$ é o instante da recuperação, e N é o número de incidentes analisados.

DORA: Para que Serve Comparar?

A pesquisa DORA (Forsgren, Humble e Kim, *Accelerate*; *State of DevOps Report*, atualizado anualmente) classifica organizações em **quatro faixas de desempenho**, cruzando as 4 métricas:

Faixa	Lead Time	Deploy Freq.	Change Fail. Rate	MTTR
Elite	minutos a horas	sob demanda (vários/dia)	baixa	minutos a horas
Alto	dias a semanas	semanal a mensal	baixa/moderada	horas a 1 dia
Médio	semanas a meses	mensal a semestral	moderada	até 1 semana
Baixo	meses	menos que semestral	alta	semanas a meses

As faixas exatas variam levemente entre edições do relatório – o que **não** muda é o padrão central da pesquisa: **velocidade e estabilidade não são opostas.** Os times de elite são, ao mesmo tempo, os mais rápidos *e* os mais estáveis.

Sumário

1. Objetivos da Aula
2. Size
   - Code Size
   - Design Size
   - Requirement Size
   - Functional Size
3. Métricas inspiradas em Normas
   - ISO/IEC 25010
   - ISO/IEC 15939
   - ISO/IEC 1061
   - IEEE Std 982.1 (1988)
   - ISO/IEC 12207
   - SPICE/ISO/IEC 15504
   - CMMI
4. Métricas Diversas
   - COCOMO
   - McCall's Quality Model
   - CK Metrics
   - Métricas de Halstead
   - Métricas de Fenton & Pfleeger (1990s, 2010s)
   - Métricas DORA
   - Team Topologies

Team Topologies (Skelton & Pais, 2019)

Abordagem organizacional para fluxo de valor, proposta a partir da **Lei de Conway**: a estrutura de um sistema tende a espelhar a estrutura de comunicação da organização que o constrói. Se a equipe está fragmentada, o sistema também fica.

O modelo define **4 tipos fundamentais de time** e **3 modos de interação** entre eles – é essa estrutura, não uma fórmula, que é o núcleo do modelo. Métricas de fluxo/colaboração vêm depois, como forma de monitorar se a topologia escolhida está funcionando.

Os 4 Tipos de Time

Stream-aligned time alinhado a um fluxo de valor de ponta a ponta (um produto, uma jornada de usuário) – é o tipo *default*, a maioria dos times deveria ser este

Enabling ajuda times *stream-aligned* a superar lacunas de capacidade (ex.: um time de práticas de teste que ensina, não que executa por eles)

Complicated-Subsystem cuida de uma parte que exige conhecimento especialista raro (ex.: um motor de otimização matemática)

Platform oferece serviços internos self-service (infraestrutura, CI/CD) que reduzem a carga cognitiva dos times *stream-aligned*

Os 3 Modos de Interação

Collaboration dois times trabalham juntos, ombro a ombro, por um período limitado – alta comunicação, mas também alto custo cognitivo

X-as-a-Service um time consome o que o outro oferece através de uma interface bem definida, com o mínimo de colaboração necessária

Facilitating um time (tipicamente *Enabling*) ajuda outro a remover impedimentos, temporariamente

O objetivo declarado do modelo é minimizar **carga cognitiva desnecessária** em cada time – é daí que vêm as métricas a seguir.

Team Topologies — medidas derivadas (não normativas)

Eficiência de Fluxo

$$\mathrm{FE} = \frac{\text{Tempo de trabalho com valor agregado}}{\text{Lead time}} \tag{62}$$

Team Topologies — medidas derivadas (não normativas)

Handoffs entre times

$$H = \sum_{\text{item}} \#\text{transferências entre times} \tag{63}$$

Team Topologies — medidas derivadas (não normativas)

Carga cognitiva aproximada (indicador composto)

$$\mathrm{CLI} = \alpha \cdot \mathrm{Dom} + \beta \cdot \mathrm{Cod} + \gamma \cdot \mathrm{Ops} \tag{64}$$

Onde: Dom, Cod, Ops = scores normalizados de domínio, código e operações; α, β, γ = pesos definidos pela organização.

Bibliografia Básica

1. FENTON, Norman; BIEMAN, James. Software metrics: a rigorous and practical approach. CRC press, 2014.
2. WOHLIN, Claes et al. Experimentation in software engineering. Berlin: Springer, 2012.
3. JURISTO, Natalia; MORENO, Ana M. Basics of software engineering experimentation. Springer Science & Business Media, 2013.
4. EMPIRICAL SOFTWARE ENGINEERING: an international journal. Boston: Kluwer Academic Publishers, 1996-. ¡20. ISSN 1573-7616. Disponível em: https://link.springer.com/journal/10664. Acesso em: 22 jul. 2024. (Periódico On-line).
5. PRESSMAN, Roger S.; MAXIM, Bruce R.. Engenharia de software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021. E-book. ISBN 9786558040118. (Livro Eletrônico).
6. WOHLIN, Claes et al. Experimentation in software engineering. New York: Springer, c2012. xxiii, 236 p. ISBN 9783642432262. (Disponível no Acervo).

Bibliografia Complementar

1. JURISTO, Natalia. Basics of Software Engineering Experimentation. 1st ed. 2001. XXII, 396 p ISBN 9781475733044. (Livro Eletrônico).
2. LAIRD, Linda M.,; BRENNAN, M. Carol,. Software measurement and estimation: a practical approach. Hoboken, N.J.: Wiley-Interscience, 2006. 1 online resource (xvi, 257 ISBN 0471792535 (electronic bk.) Disponível em : ¡https://ieeexplore.ieee.org/book/5988896¿. Acesso em: 22 jul. 2024. (Livro Eletrônico).
3. SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson Education do Brasil, c2019. xii, 756 p. ISBN 9788543024974. (Disponível no Acervo).
4. DEVORE, Jay L. Probabilidade e estatística para engenharia e ciências. 3. São Paulo Cengage Learning 2018 1 recurso online ISBN 9788522128044. (Livro Eletrônico).
5. IEEE TRANSACTIONS ON SOFTWARE ENGINEERING. New York: IEEE Computer Society ,1975-. Mensal,. ISSN 0098-5589. Disponível em: https://ieeexplore.ieee.org/xpl/RecentIssue.jsp? punumber=32. Acesso em: 22 jul. 2024. (Periódico On-line).
6. JONES, Capers. Applied Software Measurement, 3rd edition. 2008. 1 online resource (696 pages). (Livro Eletrônico).
7. SKELTON, Matthew; PAIS, Manuel. Team Topologies: Organizing Business and Technology Teams for Fast Flow. IT Revolution Press, 2019.
8. FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. Accelerate: The Science of Lean Software and DevOps. IT Revolution Press, 2018.
9. ISO/IEC 25010:2023. Systems and software engineering – SQuaRE – Product quality model.